

Deployment of 3-Tier Applications

Using AWS Services

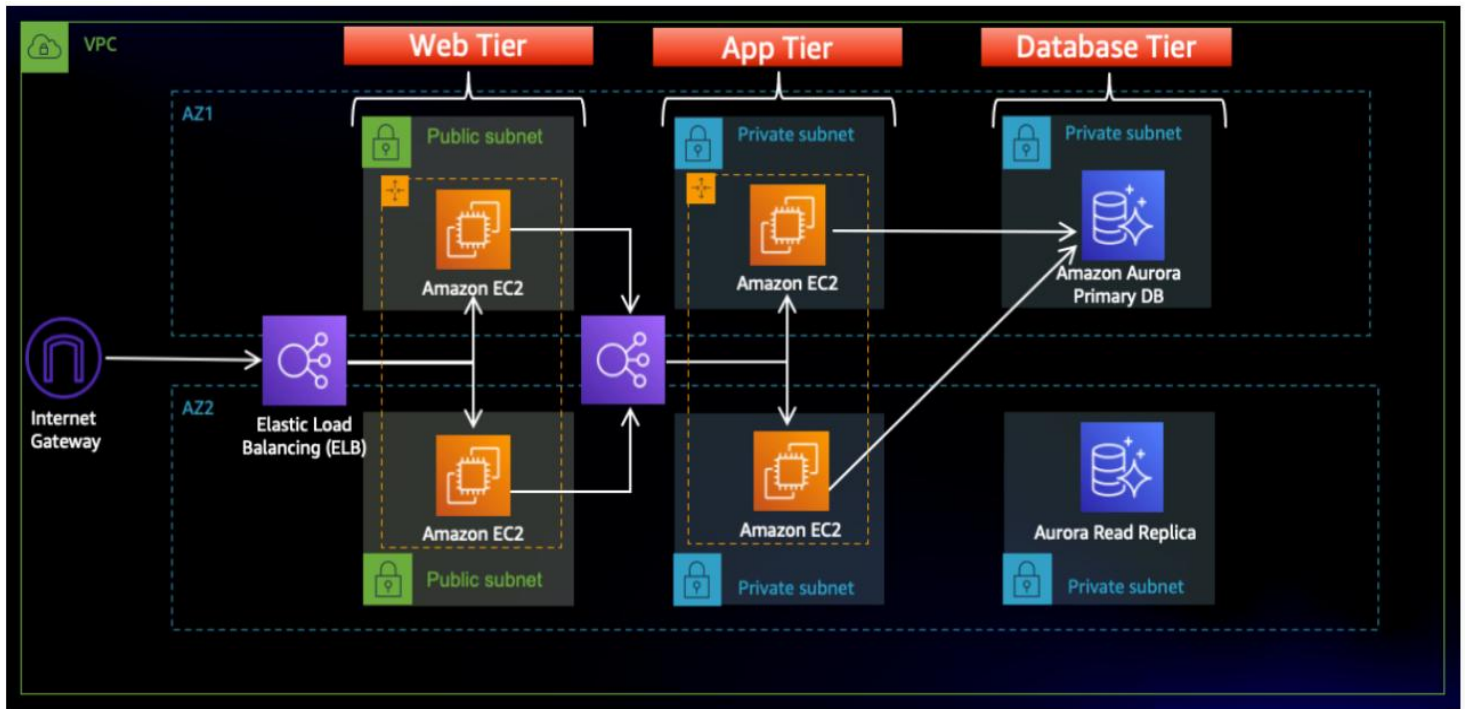
Name: Pooja Prakash Belgaumkar

This project demonstrates the deployment of a secure, scalable, and highly available 3-Tier application architecture using AWS cloud services. The architecture is divided into three tiers:

- ✓ **Web Tier** – Provides the user interface and handles incoming user requests
- ✓ **Application Tier** – Contains backend logic and processes requests from the web tier
- ✓ **Database Tier** – Stores application data securely

To ensure high availability and fault tolerance, the infrastructure is deployed across two Availability Zones within a single VPC. Public and private subnets are used to isolate resources and enhance security.

- Web tier instances are placed behind an **External Application Load Balancer** to allow access from the internet
- Application tier instances are placed behind an **Internal Load Balancer** to restrict access within the VPC
- Database tier is deployed in **private subnets** to prevent direct public access
- All resources are deployed in the same AWS region to ensure proper connectivity and performance



Repository: **3TierArchitectureApp** by PoojaPrakash08

Navigation: [Code](#) | [Pull requests](#) | [Actions](#) | [Projects](#) | [Wiki](#) | [Security](#) | [Insights](#) | [Settings](#)

Branch: **main** | Path: **3TierArchitectureApp / application-code /** | Search: Go to file | [Add file](#) | [...](#)

KastroVKiran Update Home.js · 08ca249 · 2 years ago

This branch is up to date with [KastroVKiran/3TierArchitectureApp:main](#). | [Contribute](#) | [Sync fork](#)

Name	Last commit message	Last commit date
..		
app-tier	Add files via upload	2 years ago
web-tier	Update Home.js	2 years ago
nginx-Without-SSL.conf	Add files via upload	2 years ago
nginx.conf	Add files via upload	2 years ago

1. VPC Creation

- Create the VPC named **project-vpc** with 2 public subnets (for Web Tier and External Load Balancer) and 4 private subnets (2 for App Tier, 2 for Database Tier)

The screenshot displays the AWS Management Console interface for a VPC named 'project-vpc' (ID: vpc-02aaad4b021170f04) in the Asia Pacific (Mumbai) region. The console shows the VPC's configuration details and a resource map.

VPC Details:

- VPC ID:** vpc-02aaad4b021170f04
- State:** Available
- Block Public Access:** Off
- DNS hostnames:** Enabled
- DNS resolution:** Enabled
- Tenancy:** default
- DHCP option set:** dopt-0209603241b0ef4c8
- Main network ACL:** acl-0b04fd42bd66b9b
- Default VPC:** No
- IPv4 CIDR (Network border group):** 192.168.0.0/22
- Main route table:** rtb-08b4af9c825eca7b4
- IPv6 pool:** -
- Route 53 Resolver DNS Firewall rule groups:** -
- Owner ID:** 787169320097
- Encryption control ID:** -
- Network Address Usage metrics:** Disabled
- Encryption control mode:** -

Resource map:

- VPC:** project-vpc
- Subnets (6):**
 - ap-south-1a:**
 - project-subnet-public1-ap-south-1a
 - project-subnet-private1-ap-south-1a
 - project-subnet-private3-ap-south-1a
 - ap-south-1b:**
 - project-subnet-public2-ap-south-1b
 - project-subnet-private4-ap-south-1b
 - project-subnet-private2-ap-south-1b
- Route tables (7):**
 - project-rtb-private2-ap-south-1b
 - rtb-02dd66d08911606c9
 - project-rtb-private3-ap-south-1a
 - project-rtb-private4-ap-south-1b
 - project-rtb-private1-ap-south-1a
 - project-rtb-public
 - rtb-08b4af9c825eca7b4
- Network Connections (2):**
 - project-igw
 - project-regional-nat

- Rename the subnets properly for easy identification.

☰ [VPC](#) > Subnets

Subnets (6) [Info](#) Last updated less than a minute ago [Actions](#) [Create subnet](#)

🔍 Find subnets by attribute or tag

☐	Name	Subnet ID	State	VPC
☐	project-subnet-app1-ap-south-1a	subnet-081cbdeda5d78c829	✔ Available	vpc-02aaad4b021170f04 proj...
☐	project-subnet-web2-ap-south-1b	subnet-059c78cc539f67802	✔ Available	vpc-02aaad4b021170f04 proj...
☐	project-subnet-web1-ap-south-1a	subnet-0ea31546455c7d228	✔ Available	vpc-02aaad4b021170f04 proj...
☐	project-subnet-db2-ap-south-1b	subnet-03840ad26764b7de3	✔ Available	vpc-02aaad4b021170f04 proj...
☐	project-subnet-db1-ap-south-1a	subnet-0819e839885c7acf8	✔ Available	vpc-02aaad4b021170f04 proj...
☐	project-subnet-app2-ap-south-1b	subnet-0373a114ecb983a8c	✔ Available	vpc-02aaad4b021170f04 proj...

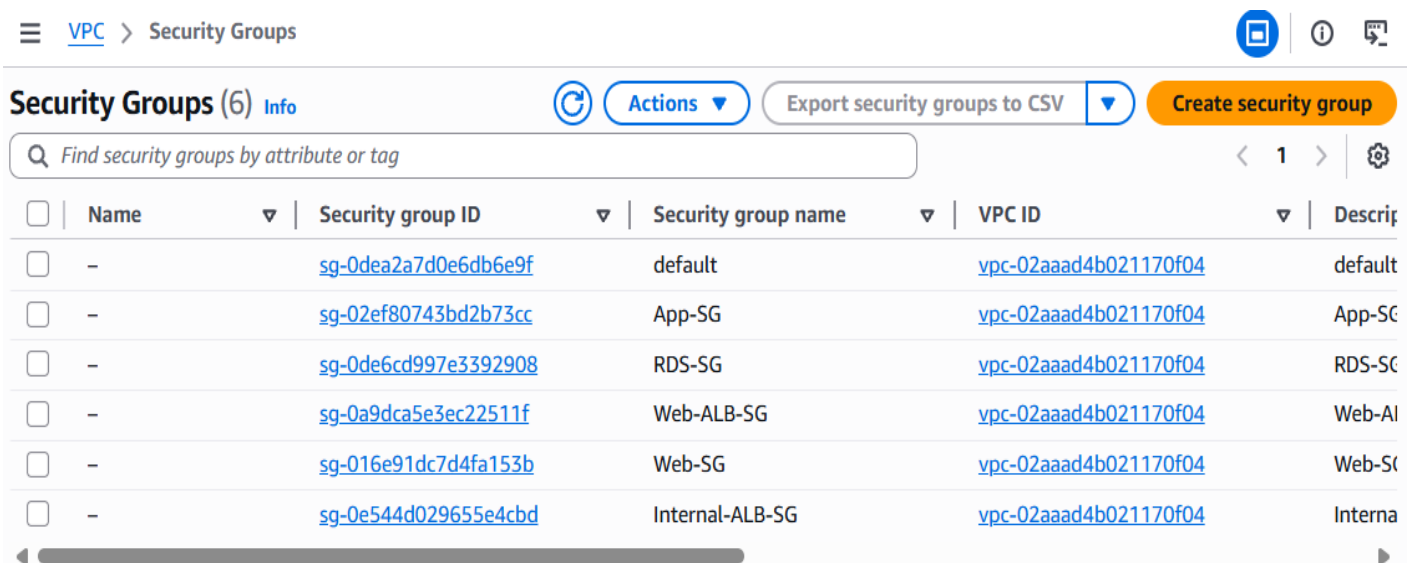
- Create the following security groups:
 - Web-ALB-SG → for External Load Balancer
 - ✓ Purpose: Allows users to access the application
 - ✓ Inbound Rules:
 - HTTP – Port 80 – Source: 0.0.0.0/0
 - Web-SG → for Web Tier Instance
 - ✓ Purpose: Allows traffic from External Load Balancer
 - ✓ Inbound Rules:
 - HTTP – Port 80 – Source: Web-ALB-SG
 - SSH (optional) – Source: VPC
 - App-SG → for Application Tier Instance
 - ✓ Purpose: Allows traffic only from Web Tier and Internal Load Balancer
 - ✓ Inbound Rules:
 - Port 4000 – Source: Internal-ALB-SG
 - SSH via SSM only

d. Internal-ALB-SG →

- ✓ Purpose: Allows communication inside VPC
- ✓ Inbound Rules:
 - i. Port 4000 – Source: Web-SG

e. RDS-SG → for Database Tier

- ✓ Purpose: Allows database access only from Application Tier
- ✓ Inbound Rules:
 - i. MySQL – Port 3306 – Source: App-SG

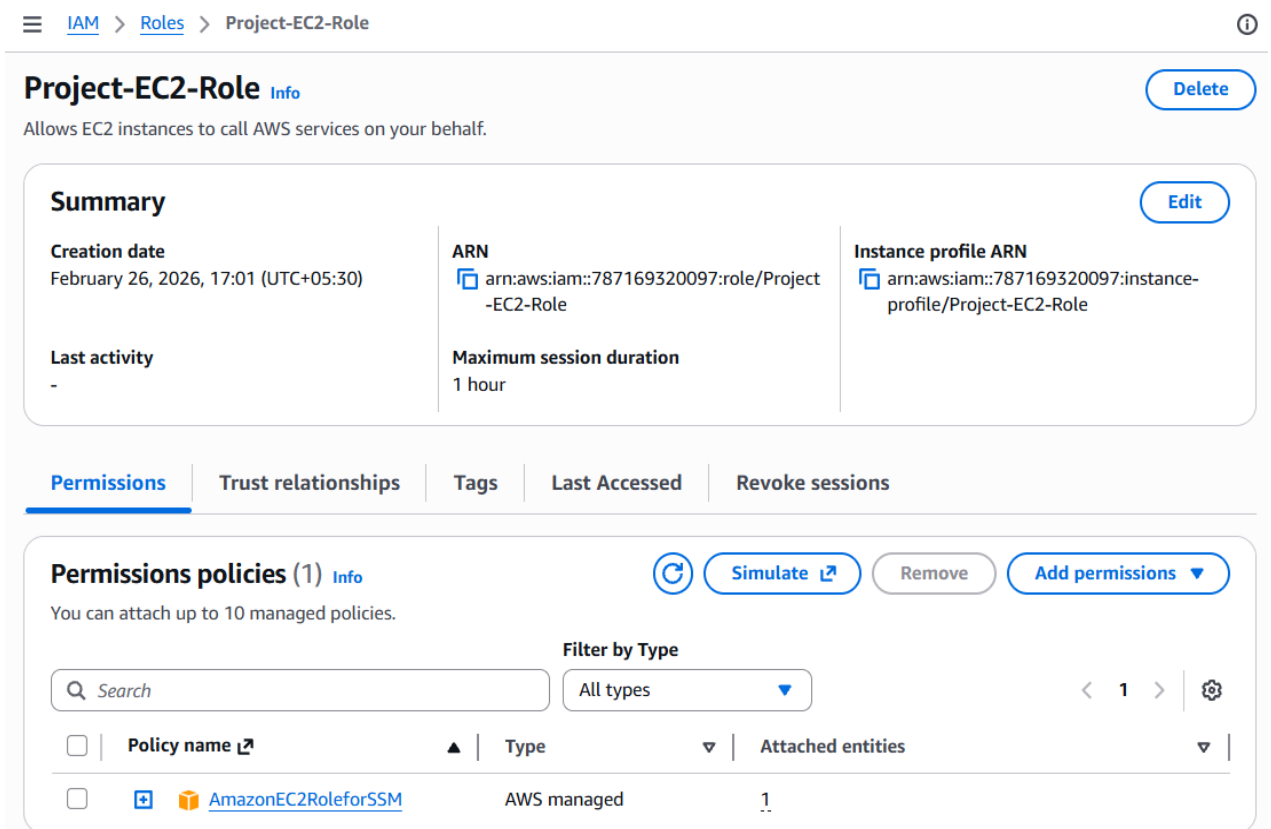
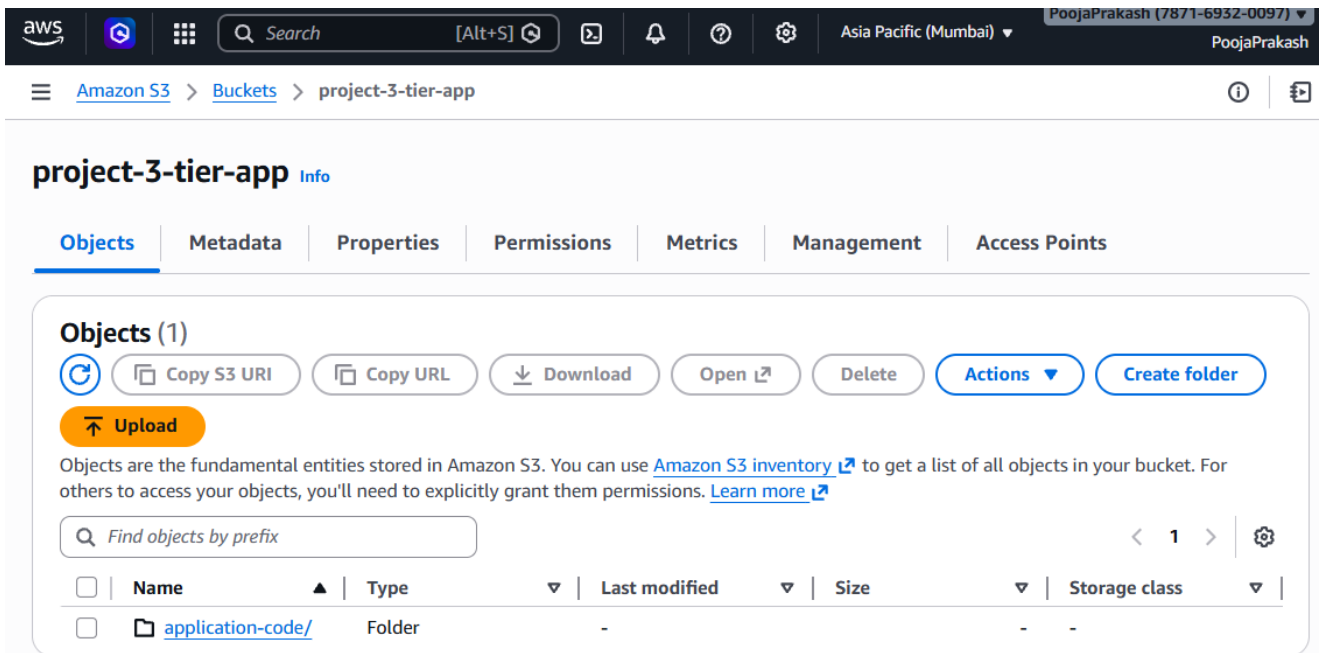


The screenshot shows the AWS Management Console 'Security Groups' page. At the top, there's a navigation bar with 'VPC' and 'Security Groups' links. Below this, the page title is 'Security Groups (6)' with an 'Info' link. There are buttons for 'Actions', 'Export security groups to CSV', and 'Create security group'. A search bar is present with the placeholder text 'Find security groups by attribute or tag'. The main content is a table listing six security groups. Each row includes a checkbox, a name (mostly dashes), a Security group ID (hyperlinked), a Security group name, a VPC ID (hyperlinked), and a Description. The security groups are: default, App-SG, RDS-SG, Web-ALB-SG, Web-SG, and Internal-ALB-SG, all associated with VPC ID vpc-02aaad4b021170f04.

☐	Name	Security group ID	Security group name	VPC ID	Description
☐	-	sg-0dea2a7d0e6db6e9f	default	vpc-02aaad4b021170f04	default
☐	-	sg-02ef80743bd2b73cc	App-SG	vpc-02aaad4b021170f04	App-SG
☐	-	sg-0de6cd997e3392908	RDS-SG	vpc-02aaad4b021170f04	RDS-SG
☐	-	sg-0a9dca5e3ec22511f	Web-ALB-SG	vpc-02aaad4b021170f04	Web-Al
☐	-	sg-016e91dc7d4fa153b	Web-SG	vpc-02aaad4b021170f04	Web-St
☐	-	sg-0e544d029655e4cbd	Internal-ALB-SG	vpc-02aaad4b021170f04	Interna

2. S3 Bucket and IAM Role Setup

- Directly uploading the source code into instance may hassle for further process thus we will create the s3 bucket with named **project-3-tier-app** in the same region and upload the application source in this bucket
- This allows EC2 instances to securely download application code.



3. Database Configuration

- Now we will create subnet group using only Private DB subnets to ensures database is accessible only from private network.

☰ [Aurora and RDS](#) > [Subnet groups](#) > db-subnetgroup ⓘ 🖨

db-subnetgroup

Subnet group details

VPC ID
[vpc-02aaad4b021170f04](#) 🔗

ARN
arn:aws:rds:ap-south-1:787169320097:subgrp:db-subnetgroup

Supported network types
IPv4

Description
DB-SubnetGroup

Subnets (2)

Availability zone	Subnet name	Subnet ID	CIDR block
ap-south-1b	project-subnet-db2-ap-south-1b	subnet-03840ad26764b7de3 🔗	192.168.2.192/26
ap-south-1a	project-subnet-db1-ap-south-1a	subnet-0819e839885c7acf8 🔗	192.168.2.128/26

- Then create the RDS Database with configuration:
 - Engine: MySQL
 - Deployment: Private subnets
 - Security Group: RDS-SG
 - Public access: Disabled
- It provides secure and managed database service.

project-db

Refresh Refresh Modify Actions

Summary

DB identifier project-db	Status Available	Role Instance	Engine MySQL Community
CPU 4.59%	Class db.t4g.micro	Current activity 0 Connections	Region & AZ ap-south-1b
Recommendations			

< Connectivity & security Monitoring Logs & events Configuration Zero-ETL integrations M >

Connect using Info

- ☐ Code snippets
Use when connecting through SDK, APIs, or third-party tools including agents.
- ☐ CloudShell
Use for a quick access to AWS CLI that launches directly from the AWS Management Console.
- ☒ Endpoints
Use when connecting through any IDE interface.

Database name mysql	Master username admin	Internet access gateway Disabled	Port 3306
-------------------------------	---------------------------------	--	---------------------

Additional configurations

Connectivity & security

Endpoint & port

Endpoint
project-db.c1u60cimaz7w.ap-south-1.rds.amazonaws.com

Port
3306

Networking

Availability Zone
ap-south-1b

VPC
project-vpc (vpc-02aaad4b021170f04)

Subnet group
db-subnetgroup

Subnets
subnet-0819e839885c7acf8
subnet-03840ad26764b7de3

Network type
IPv4

Security

VPC security groups
RDS-SG (sg-0de6cd997e3392908)
Active

Publicly accessible
No

Certificate authority Info
rds-ca-rsa2048-g1

Certificate authority date
May 20, 2061, 00:10 (UTC+05:30)

DB instance certificate expiration date
February 26, 2027, 17:18 (UTC+05:30)

☰ [Aurora and RDS](#) > [Databases](#) > [project-db](#) ⓘ 🗨

no connected compute resources that were created automatically to display.

[Set up EC2 connection](#) [Set up Lambda connection](#)

Security group rules (2)

🔍 < 1 > ⚙

Security group ▲	Type ▼	Rule ▼
RDS-SG (sg-0de6cd997e3392908)	CIDR/IP - Inbound	192.168.0.0/22
RDS-SG (sg-0de6cd997e3392908)	CIDR/IP - Outbound	0.0.0.0/0

Replication (1)

🔍 < 1 > ⚙

DB identifier ▲	Role ▼	Region & AZ ▼	Replication source ▼	Replication state ▼	Lag ▼
project-db	Instance	ap-south-1b	-	-	-

4. Application Tier Setup

- Create App Tier EC2 Instance named **App-Tier-Instance** inside the app tier subnet and connect it to instance through SSM session management as the subnet is private



Instance summary for i-07640aa6bce4d8834 (App-Tier-Instance) [Info](#)

Connect

Instance state ▼

Actions ▼

Updated less than a minute ago

Instance ID i-07640aa6bce4d8834	Public IPv4 address -	Private IPv4 addresses 192.168.2.22
IPv6 address -	Instance state Running	Public DNS -
Hostname type IP name: ip-192-168-2-22.ap-south-1.compute.internal	Private IP DNS name (IPv4 only) ip-192-168-2-22.ap-south-1.compute.internal	
Answer private resource DNS name -	Instance type t3.micro	Elastic IP addresses -
Auto-assigned IP address -	VPC ID vpc-02aaad4b021170f04 (project-vpc)	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more
IAM role Project-EC2-Role	Subnet ID subnet-081cbdeda5d78c829 (project-subnet-app1-ap-south-1a)	Auto Scaling Group name -



Connect [Info](#)

Connect to an instance using the browser-based client.

Instance ID i-07640aa6bce4d8834 (App-Tier-Instance)	VPC ID vpc-02aaad4b021170f04	Security groups sg-02ef80743bd2b73cc (App-SG)	IAM role Project-EC2-Role
---	--	---	-------------------------------------

EC2 Instance Connect | **SSM Session Manager** | SSH client | EC2 serial console

SSM agent status

Ping status Online	Last ping time Thu Feb 26 2026 17:43:32 GMT+0530 (India Standard Time)	Session Manager connection status Connected	Agent version 3.3.3598.0
------------------------------	--	---	------------------------------------

Session Manager usage:

- Connect to your instance without SSH keys, a bastion host, or opening any inbound ports.
- Sessions are secured using an AWS Key Management Service key.
- You can log session commands and details in an Amazon S3 bucket or CloudWatch Logs log group.
- Configure sessions on the Session Manager [Preferences](#) page.

- Inside the App-Tier-Instance ec2-user we will install the MySQL to communicate with the database
- Connect to the database using the endpoint of Database and perform basic configuration

```
[root@ip-192-168-2-22 ec2-user]# mysql --version
mysql Ver 15.1 Distrib 10.5.29-MariaDB, for Linux (x86_64) using EditLine wrapper
[root@ip-192-168-2-22 ec2-user]# mysql -h project-db.c1u60cimaz7w.ap-south-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 46
Server version: 8.0.40 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> █
```

- Create the **webapp** Database and inside the Database create table add sample data.

```
MySQL [(none)]> CREATE DATABASE webappdb;
Query OK, 1 row affected (0.022 sec)

MySQL [(none)]> SHOW DATABASES;
+-----+
| Database                |
+-----+
| information_schema      |
| mysql                   |
| performance_schema      |
| sys                     |
| webappdb                |
+-----+
5 rows in set (0.011 sec)

MySQL [(none)]> USE webappdb;
Database changed
MySQL [webappdb]> █
```

```
MySQL [webappdb]> CREATE TABLE IF NOT EXISTS transactions(
  ->   id INT NOT NULL AUTO_INCREMENT,
  ->   amount DECIMAL(10,2),
  ->   description VARCHAR(100),
  ->   PRIMARY KEY(id)
  -> );
```

Query OK, 0 rows affected (0.064 sec)

```
MySQL [webappdb]> SHOW TABLES;
```

```
+-----+
| Tables_in_webappdb |
+-----+
| transactions        |
+-----+
1 row in set (0.002 sec)
```

```
MySQL [webappdb]>
```

```
MySQL [webappdb]> INSERT INTO transactions (amount, description) VALUES ('400', 'groceries');
Query OK, 1 row affected (0.006 sec)
```

```
MySQL [webappdb]> SELECT * FROM transactions;
```

```
+-----+-----+-----+
| id | amount | description |
+-----+-----+-----+
| 1 | 400.00 | groceries  |
+-----+-----+-----+
1 row in set (0.002 sec)
```

```
MySQL [webappdb]> exit
```

Bye

```
[root@ip-192-168-2-22 ec2-user]#
```

- Update the Application Configuration to with our created database credentials and replace the file in the s3 bucket with new updated file

```
JS DbConfig.js M X
dy Materials > GitHub Portfolio > Projects > 3-Tier Architecture Application > 3TierArchitectureApp > application-code > app-tier > JS DbConfig.js > .
1  module.exports = Object.freeze({
2    DB_HOST : 'project-db.c1u60cimaz7w.ap-south-1.rds.amazonaws.com',
3    DB_USER : 'admin',
4    DB_PWD  : 'pooja123',
5    DB_DATABASE : 'webappdb'
6  });
7
```

Upload: status

Close

After you navigate away from this page, the following information is no longer available.

Summary

Destination

s3://project-3-tier-app/application-code/app-tier/

Succeeded

✓ 1 file, 190.0 B (100.00%)

Failed

✗ 0 files, 0 B (0%)

Files and folders

Configuration

Files and folders (1 total, 190.0 B)

< 1 >

Name	Folder	Type	Size	Status	Error
DbConfig.js	-	text/javascript	190.0 B	✓ Succeeded	-

- Install Node.js and PM2 process manager
 - Purpose: Node.js runs application and PM2 keeps application running continuously

```
[root@ip-192-168-2-22 ec2-user]# curl -o- https://raw.githubusercontent.com/avizway1/aws_3tier_architecture/main/install.sh | bash
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100 14926 100 14926    0     0 46952      0  --:--:-- --:--:-- --:--:-- 46937
=> Downloading nvm as script to '/root/.nvm'

=> Appending nvm source string to /root/.bashrc
=> Appending bash_completion source string to /root/.bashrc
=> Close and reopen your terminal to start using nvm or run the following to use it now:

export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh" # This loads nvm
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion" # This loads nvm bash_completion
[root@ip-192-168-2-22 ec2-user]# source ~/.bashrc
[root@ip-192-168-2-22 ec2-user]# nvm install 16
Downloading and installing node v16.20.2...
Downloading https://nodejs.org/dist/v16.20.2/node-v16.20.2-linux-x64.tar.xz...
##### 100.0%
Computing checksum with sha256sum
Checksums matched!
Now using node v16.20.2 (npm v8.19.4)
Creating default alias: default -> 16 (-> v16.20.2)
[root@ip-192-168-2-22 ec2-user]# nvm use 16
Now using node v16.20.2 (npm v8.19.4)
[root@ip-192-168-2-22 ec2-user]#
```

```
[root@ip-192-168-2-22 ec2-user]# npm install -g pm2
added 133 packages, and audited 134 packages in 10s

13 packages are looking for funding
  run `npm fund` for details

1 low severity vulnerability

Some issues need review, and may require choosing
a different dependency.

Run `npm audit` for details.
npm notice
npm notice New major version of npm available! 8.19.4 -> 11.11.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.11.0
npm notice Run npm install -g npm@11.11.0 to update!
npm notice
[root@ip-192-168-2-22 ec2-user]#
```

- Download application code from S3 bucket and start the application.

```
[root@ip-192-168-2-22 ~]# pwd
/root
[root@ip-192-168-2-22 ~]# sudo aws s3 cp s3://project-3-tier-app/application-code/app-tier/ app-tier -
-recurisve
download: s3://project-3-tier-app/application-code/app-tier/DbConfig.js to app-tier/DbConfig.js
download: s3://project-3-tier-app/application-code/app-tier/README.md to app-tier/README.md
download: s3://project-3-tier-app/application-code/app-tier/TransactionService.js to app-tier/Transact
ionService.js
download: s3://project-3-tier-app/application-code/app-tier/package-lock.json to app-tier/package-lock
.json
download: s3://project-3-tier-app/application-code/app-tier/package.json to app-tier/package.json
download: s3://project-3-tier-app/application-code/app-tier/index.js to app-tier/index.js
[root@ip-192-168-2-22 ~]# ls
app-tier
```

- Inside the app-tier folder we have inde.js file we have to start this file using pm2 start index.js command then we will see status as online.

```
[root@ip-192-168-2-22 ~]# cd app-tier
[root@ip-192-168-2-22 app-tier]# npm install
```

added 68 packages, and audited 69 packages in 2s

```
2 packages are looking for funding
  run `npm fund` for details
```

7 vulnerabilities (3 low, 4 high)

```
To address all issues, run:
  npm audit fix
```

Run ``npm audit`` for details.

```
[root@ip-192-168-2-22 app-tier]# ls
DbConfig.js  TransactionService.js  node_modules  package.json
README.md    index.js               package-lock.json
[root@ip-192-168-2-22 app-tier]#
```

```
[root@ip-192-168-2-22 app-tier]# ls
DbConfig.js  TransactionService.js  node_modules  package.json
README.md    index.js               package-lock.json
[root@ip-192-168-2-22 app-tier]# pm2 start index.js
```

```
[root@ip-192-168-2-22 app-tier]# pm2 status
```

id	name	mode	0	status	cpu	memory
0	index		0	online	0%	51.9mb

```
[root@ip-192-168-2-22 app-tier]# pm2 logs
```

```
[TAILING] Tailing last 15 lines for [all] processes (change the value with --lines option)
```

```
/root/.pm2/pm2.log last 15 lines:
```

```
PM2 | 2026-02-26T12:54:52: PM2 log: Node.js version      : 16.20.2
PM2 | 2026-02-26T12:54:52: PM2 log: Current arch       : x64
PM2 | 2026-02-26T12:54:52: PM2 log: PM2 home            : /root/.pm2
PM2 | 2026-02-26T12:54:52: PM2 log: PM2 PID file         : /root/.pm2/pm2.pid
PM2 | 2026-02-26T12:54:52: PM2 log: RPC socket file      : /root/.pm2/rpc.sock
PM2 | 2026-02-26T12:54:52: PM2 log: BUS socket file      : /root/.pm2/pub.sock
PM2 | 2026-02-26T12:54:52: PM2 log: Application log path  : /root/.pm2/logs
PM2 | 2026-02-26T12:54:52: PM2 log: Worker Interval      : 30000
PM2 | 2026-02-26T12:54:52: PM2 log: Process dump file    : /root/.pm2/dump.pm2
PM2 | 2026-02-26T12:54:52: PM2 log: Concurrent actions   : 2
PM2 | 2026-02-26T12:54:52: PM2 log: SIGTERM timeout      : 1600
PM2 | 2026-02-26T12:54:52: PM2 log: Runtime Binary       : /root/.npm/versions/node/v16.20.2/bin/node
```

 n/node

```
PM2 | 2026-02-26T12:54:52: PM2 log: =====
PM2 | 2026-02-26T12:54:52: PM2 log: App [index:0] starting in -fork mode-
PM2 | 2026-02-26T12:54:52: PM2 log: App [index:0] online
```

```
/root/.pm2/logs/index-error.log last 15 lines:
```

```
/root/.pm2/logs/index-out.log last 15 lines:
```

```
0|index      | AB3 backend app listening at http://localhost:4000
```

- Start the application using pm2 commands and verify `curl http://localhost:4000/health` the expected output should be “This is the health check”

```
[PM2] [v] Command successfully executed.
+-----+
[PM2] Freeze a process list on reboot via:
$ pm2 save

[PM2] Remove init script via:
$ pm2 unstartup systemd
[root@ip-192-168-2-22 app-tier]# pm2 save
[PM2] Saving current process list...
[PM2] Successfully saved in /root/.pm2/dump.pm2
[root@ip-192-168-2-22 app-tier]# curl http://localhost:4000/health
"This is the health check"[root@ip-192-168-2-22 app-tier]#
```

- Create the target group named **App-Internal-TG** to allow routes traffic from internal load balancer to application instance.

EC2 > Target groups > App-Internal-TG

App-Internal-TG Actions ▾

Details

arn:aws:elasticloadbalancing:ap-south-1:787169320097:targetgroup/App-Internal-TG/de75ae518a7e83b8

Target type Instance	Protocol : Port HTTP: 4000	Protocol version HTTP1	VPC vpc-02aad4b021170f04
IP address type IPv4	Load balancer None associated		

1 Total targets	✓ 0 Healthy 0 Anomalous	✗ 0 Unhealthy	⋮ 1 Unused	⌚ 0 Initial	⌚ 0 Draining
--------------------	--	--	---	--	---

► **Distribution of targets by Availability Zone (AZ)**
Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets

Monitoring

Health checks

Attributes

Tags

Registered targets (1) Info

Anomaly mitigation: Not applicable

Deregister

Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

< 1 > ⚙️

☐	Instance ID	Name	Port	Zone	Health status
☐	i-07640aa6bce4d8834	App-Tier-Instance	4000	ap-south-1a (aps1-az1)	Unused

- Then create the load balancer named **App-internal-LB** configures the private subnets and select the App-Internal-TG target group.
- It allows secure communication between Web Tier and App Tier.

EC2 > Load balancers > App-Internal-LB

App-Internal-LB

⌂ Actions

▼ Details

Load balancer type Application	Status Active	VPC vpc-02aad4b021170f04	Load balancer IP address type IPv4
Scheme Internal	Hosted zone ZP97RAFLXTNZK	Availability Zones subnet-081cbdeda5d78c829 ap-south-1a (aps1-az1) subnet-0373a114ecb983a8c ap-south-1b (aps1-az3)	Date created February 26, 2026, 18:35 (UTC+05:30)
Load balancer ARN arn:aws:elasticloadbalancing:ap-south-1:787169320097:loadbalancer/app/App-Internal-LB/5ad5e87c9769f72a		DNS name Info internal-App-Internal-LB-1974454396.ap-south-1.elb.amazonaws.com (A Record)	

Listeners and rules

Network mapping

Resource map

Security

Monitoring

Integrations

Listeners and rules (1) Info

Manage rules

Manage listener

Add listener

A listener checks for connection requests on its configured protocol and port. Traffic received by the listener is routed according to the default action and any additional rules.

< 1 > ⚙️

☐	Protocol:Port	Default action	Rules	ARN	Security policy
☐	HTTP:80	• Forward to target group App-Internal-TG : 1 (100%) Target group stickiness: Off	1 rule	ARN	Not applicable

- Update the configuration with Internal Load Balancer DNS and update the file in s3 bucket. It ensures Web Tier connects to Application Tier securely.

```
JS DbConfig.js M X  nginx.conf M X
OJA > Desktop > DevOps Study Materials > GitHub Portfolio > Projects > 3-Tier Architecture Application > 3TierArchitectureApp > application-code > nginx.conf
13 http {
31     server {
46
47         #react app and front end files
48         location / {
49             root    /home/ec2-user/web-tier/build;
50             index index.html index.htm;
51             try_files $uri /index.html;
52         }
53
54         #proxy for internal lb
55         location /api/ {
56             proxy_pass http://internal-App-Internal-LB-1974454396.ap-south-1.elb.amazonaws.com;
57         }
58     }
59
60 }
```

Amazon S3

Upload: status

After you navigate away from this page, the following information is no longer available.

Summary

Destination s3://project-3-tier-app/application-code/	Succeeded ✔ 1 file, 1.6 KB (100.00%)	Failed ✖ 0 files, 0 B (0%)
---	--	--------------------------------------

Files and folders | Configuration

Files and folders (1 total, 1.6 KB)

Find by name

Name	Folder	Type	Size	Status	Error
nginx.conf	-	-	1.6 KB	✔ Succeeded	-

5. Web Tier Setup

- Create web tier EC2 Instance **Web-Tier-Instance** in the public subnet and access it using SSM Session Management.

EC2 > Instances > i-06eb51802681d0e9f

Instance summary for i-06eb51802681d0e9f (Web-Tier-Instance) Info

Connect

Instance state ▼

Actions ▼

Updated less than a minute ago

Instance ID i-06eb51802681d0e9f	Public IPv4 address 52.66.202.85 open address	Private IPv4 addresses 192.168.0.46
IPv6 address -	Instance state Running	Public DNS ec2-52-66-202-85.ap-south-1.compute.amazonaws.com open address
Hostname type IP name: ip-192-168-0-46.ap-south-1.compute.internal	Private IP DNS name (IPv4 only) ip-192-168-0-46.ap-south-1.compute.internal	Elastic IP addresses -
Answer private resource DNS name -	Instance type t3.micro	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto-assigned IP address 52.66.202.85 [Public IP]	VPC ID vpc-02aaad4b021170f04 (project-vpc)	Auto Scaling Group name -
IAM role Project-EC2-Role	Subnet ID subnet-0ea31546455c7d228 (project-subnet-web1-ap-south-1a)	

EC2 > Instances > i-06eb51802681d0e9f > Connect to instance

Instance ID
i-06eb51802681d0e9f (Web-Tier-Instance)

VPC ID
vpc-02aaad4b021170f04

Security groups
sg-016e91dc7d4fa153b (Web-SG)

IAM role
Project-EC2-Role

EC2 Instance Connect

SSM Session Manager

SSH client

EC2 serial console

SSM agent status

Ping status Online	Last ping time Thu Feb 26 2026 19:25:30 GMT+0530 (India Standard Time)	Session Manager connection status Connected	Agent version 3.3.3598.0
------------------------------	--	---	------------------------------------

Session Manager usage:

- Connect to your instance without SSH keys, a bastion host, or opening any inbound ports.
- Sessions are secured using an AWS Key Management Service key.
- You can log session commands and details in an Amazon S3 bucket or CloudWatch Logs log group.
- Configure sessions on the Session Manager [Preferences](#) page.

Cancel

Connect

- Install Node.js and Nginx

Session ID: root-456altf2dnnryjbqi9trl8btqu [Shortcuts](#) Instance ID: i-06eb51802681d0e9f [Terminate](#)

```
sh-5.2$ sudo -su ec2-user
[ec2-user@ip-192-168-0-46 bin]$ whoami
ec2-user
[ec2-user@ip-192-168-0-46 bin]$ cd /home/ec2-user
[ec2-user@ip-192-168-0-46 ~]$ curl -o- https://raw.githubusercontent.com/avizway1/aws_3tier_architecture/main/install.sh | bash
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 14926 100 14926    0     0 54205      0 --:--:-- --:--:-- --:--:-- 54276
=> Downloading nvm as script to '/home/ec2-user/.nvm'

=> Appending nvm source string to /home/ec2-user/.bashrc
=> Appending bash_completion source string to /home/ec2-user/.bashrc
=> Close and reopen your terminal to start using nvm or run the following to use it now:

export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh" # This loads nvm
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion" # This loads nvm bash_completion
[ec2-user@ip-192-168-0-46 ~]$ source ~/.bashrc

[ec2-user@ip-192-168-0-46 ~]$ nvm install 16
Downloading and installing node v16.20.2...
Downloading https://nodejs.org/dist/v16.20.2/node-v16.20.2-linux-x64.tar.xz...
##### 100.0%
Computing checksum with sha256sum
Checksums matched!
Now using node v16.20.2 (npm v8.19.4)
Creating default alias: default -> 16 (-> v16.20.2)
[ec2-user@ip-192-168-0-46 ~]$ nvm use 16
Now using node v16.20.2 (npm v8.19.4)
[ec2-user@ip-192-168-0-46 ~]$
```

- Download the web page related code from s3 and start the application.

Session ID: root-456altf2dnnryjbqi9trl8btqu [Shortcuts](#) Instance ID: i-06eb51802681d0e9f [Terminate](#)

```
[ec2-user@ip-192-168-0-46 ~]$ aws s3 cp s3://project-3-tier-app/application-code/web-tier/ web-tier --recursive
download: s3://project-3-tier-app/application-code/web-tier/README.md to web-tier/README.md
download: s3://project-3-tier-app/application-code/web-tier/src/components/Menu/Menu.js to web-tier/src/components/Menu/Menu.js
download: s3://project-3-tier-app/application-code/web-tier/public/robots.txt to web-tier/public/robots.txt
download: s3://project-3-tier-app/application-code/web-tier/package.json to web-tier/package.json
download: s3://project-3-tier-app/application-code/web-tier/src/components/Menu/Menu.styled.js to web-tier/src/components/Menu/Menu.styled.js
download: s3://project-3-tier-app/application-code/web-tier/src/components/Burger/index.js to web-tier/src/components/Burger/index.js
download: s3://project-3-tier-app/application-code/web-tier/public/index.html to web-tier/public/index.html
```

```
[ec2-user@ip-192-168-0-46 ~]$ ls
web-tier
[ec2-user@ip-192-168-0-46 ~]$ cd web-tier
[ec2-user@ip-192-168-0-46 web-tier]$ npm install
npm WARN EBADENGINE Unsupported engine {
npm WARN EBADENGINE   package: '@testing-library/dom@10.4.1',
npm WARN EBADENGINE   required: { node: '>=18' },
npm WARN EBADENGINE   current: { node: 'v16.20.2', npm: '8.19.4' }
npm WARN EBADENGINE }
```

- Update nginx.conf file and set permissions so it allows Nginx to act as reverse proxy to application tier.

```
[ec2-user@ip-192-168-0-46 nginx]$ sudo dnf install nginx -y
Last metadata expiration check: 0:15:27 ago on Thu Feb 26 14:23:40 2026.
Package nginx-1:1.28.2-1.amzn2023.0.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-192-168-0-46 nginx]$ cd /etc/nginx
[ec2-user@ip-192-168-0-46 nginx]$ ls
conf.d          fastcgi_params  mime.types      scgi_params     win-utf
default.d       fastcgi_params.default  mime.types.default  scgi_params.default
fastcgi.conf    koi-utf        nginx.conf      uwsgi_params
fastcgi.conf.default  koi-win      nginx.conf.default  uwsgi_params.default
[ec2-user@ip-192-168-0-46 nginx]$ sudo rm nginx.conf
[ec2-user@ip-192-168-0-46 nginx]$ sudo aws s3 cp s3://project-3-tier-app/application-code/nginx.conf .
download: s3://project-3-tier-app/application-code/nginx.conf to ./nginx.conf
[ec2-user@ip-192-168-0-46 nginx]$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
[ec2-user@ip-192-168-0-46 nginx]$ sudo systemctl restart nginx
[ec2-user@ip-192-168-0-46 nginx]$
```

```
[ec2-user@ip-192-168-0-46 nginx]$ chmod -R 755 /home/ec2-user
[ec2-user@ip-192-168-0-46 nginx]$ sudo systemctl enable nginx
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr/lib/systemd/system/nginx.service.
[ec2-user@ip-192-168-0-46 nginx]$
```

- Create the external target group named **External-WebTG** for Web-Tier-Instance

EC2 > Target groups > External-WebTG

External-WebTG Actions

Details
arn:aws:elasticloadbalancing:ap-south-1:787169320097:targetgroup/External-WebTG/ca5b9f27c6bf8ed7

Target type Instance	Protocol : Port HTTP: 80	Protocol version HTTP1	VPC vpc-02aaad4b021170f04
IP address type IPv4	Load balancer None associated		

1 Total targets	0 Healthy	0 Unhealthy	1 Unused	0 Initial	0 Draining
0 Anomalous					

► **Distribution of targets by Availability Zone (AZ)**
Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets | Monitoring | Health checks | Attributes | Tags

Registered targets (1) [Info](#) [Anomaly mitigation: Not applicable](#) [Deregister](#) [Register targets](#)

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

☐	Instance ID	Name	Port	Zone	Health status	Health status
☐	i-06eb51802681d0e9f	Web-Tier-Insta...	80	ap-south-1a (a...	Unused	Target grou

- Create the External Load Balancer named **External-Web-ALB** with External-WebTG in web tier subnets.
- We can access the Web page using the DNS name of the load balancer and also public ip of the web-tier-instance with port 80.

[EC2](#) > [Load balancers](#) > External-Web-ALB

External-Web-ALB [Refresh](#) [Actions](#)

▼ Details

Load balancer type Application	Status ✔ Active	VPC vpc-02aaad4b021170f04	Load balancer IP address type IPv4
Scheme Internet-facing	Hosted zone ZP97RAFLXTNZK	Availability Zones subnet-059c78cc539f67802 ap-south-1b (aps1-az3) subnet-0ea31546455c7d228 ap-south-1a (aps1-az1)	Date created February 26, 2026, 20:29 (UTC+05:30)

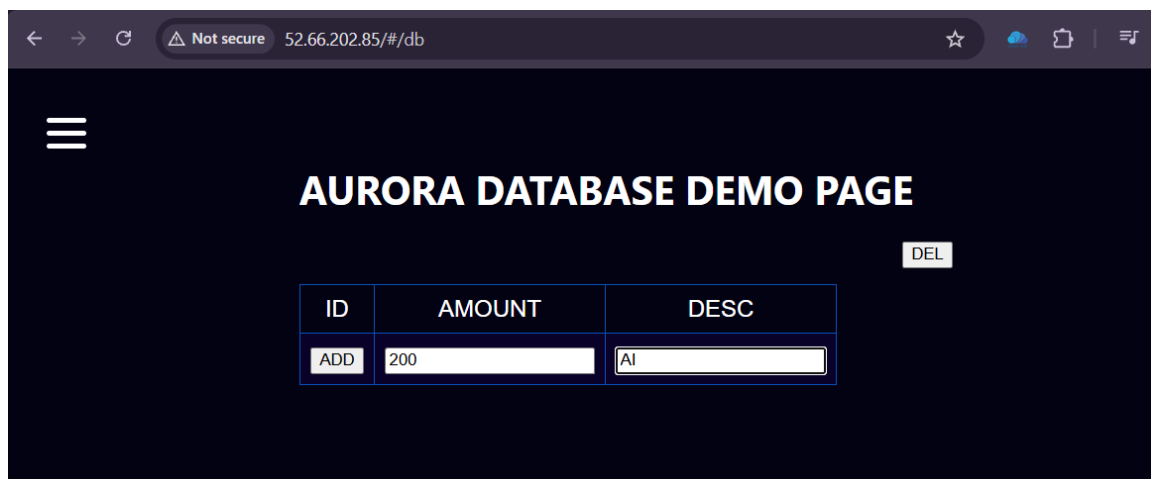
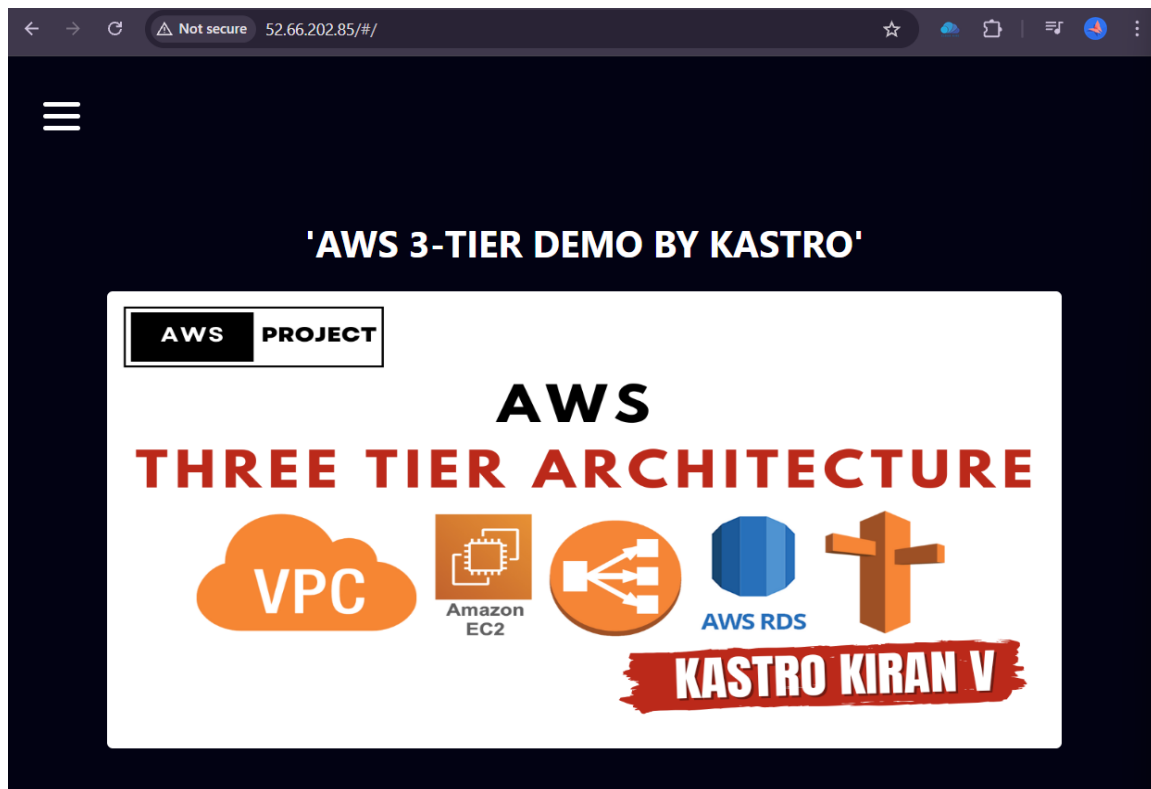
Load balancer ARN arn:aws:elasticloadbalancing:ap-south-1:787169320097:loadbalancer/app/External-Web-ALB/e65fe74527e2a480	DNS name Info External-Web-ALB-580055460.ap-south-1.elb.amazonaws.com (A Record)
---	--

[Listeners and rules](#) | Network mapping | Resource map | Security | Monitoring | [Inte](#)

Listeners and rules (1) [Info](#) [Refresh](#) [Manage rules](#) [Manage listener](#) [Add listener](#)

A listener checks for connection requests on its configured protocol and port. Traffic received by the listener is routed according to the default action and any additional rules.

☐	Protocol:Port	Default action	Rules	ARN	Security policy
☐	HTTP:80	• Forward to target group External-WebTG: 1 (100%) Target group stickiness: Off	1 rule	ARN	Not applicable



- This project demonstrates the successful deployment of a secure and highly available 3-Tier application architecture using AWS services such as VPC, EC2, RDS, S3, IAM and Load Balancers. By separating the web, application and database tiers into public and private subnets and using internal and external load balancers, the architecture ensures proper security, scalability and efficient communication between components. This implementation reflects practical experience in AWS infrastructure design, networking and deploying real-world production-style applications.